SuperMinHash paso a paso

Ejemplo de construcción de una firma

1. Qué vamos a calcular

SuperMinHash resume un conjunto de elementos en una **firma** de tamaño m. Si $m = 3$, la firma tiene tres posiciones:

$$h = [h_0, h_1, h_2].$$

Cada posición conserva el menor valor que se le haya asignado. En una iteración con índice j, se genera un candidato $r + j$, donde r es un número aleatorio entre 0 y 1. Una permutación aleatoria determina a qué posición de la firma se asigna ese candidato.

Importante: los valores que siguen son una ejecución inventada para explicar el mecanismo. No son una salida fija del algoritmo.

2. Procesamiento del primer elemento

La firma parte sin valores finitos: $h = [\infty, \infty, \infty]$. Para simplificar, mostramos directamente qué posición selecciona la permutación en cada paso.

Iteración j	Aleatorio r	Candidato $r + j$	Posición	Valor previo	Valor nuevo	Acción
0	0,4	0,4	h_2	∞	0,4	Actualiza
1	0,2	1,2	h_0	∞	1,2	Actualiza
2	0,7	2,7	h_1	∞	2,7	Actualiza

La firma queda $h = [1,2;\ 2,7;\ 0,4]$. Se hicieron las tres iteraciones posibles. La parte entera de sus valores es, respectivamente, 1, 2 y 0.

3. El histograma permite cortar el segundo elemento

El algoritmo mantiene un histograma con la cantidad de posiciones de la firma que tienen cada parte entera. La variable a guarda la parte entera más alta que todavía aparece. Después del primer elemento:

Firma actual	Conteo de parte entera 0	Conteo de 1	Conteo de 2
$[1,2;\ 2,7;\ 0,4]$	1	1	1

$a = 2$

Para el segundo elemento:

1. En $j = 0$, el candidato es 0,1 y se asigna a h_1. Como $0,1 < 2,7$, actualiza la firma a $[1,2;\ 0,1;\ 0,4]$.

2. La parte entera 2 ya no aparece: las partes enteras son ahora $[1, 0, 0]$. El histograma lo refleja y baja a de 2 a 1.

3. En $j = 1$, el candidato es 1,6 y se asigna a h_2. No actualiza, porque $1,6 > 0,4$.

4. La siguiente iteración sería $j = 2$. Como $j > a$ (2>1), el algoritmo se detiene para este elemento.

¿Por qué puede parar? Cuando $j = 2$, cualquier candidato es al menos 2. Todas las posiciones actuales son menores que 2, así que ningún candidato restante podría reemplazar un mínimo por uno menor.

4. Un tercer elemento muestra un corte más temprano

La firma actual es $[1,2;\ 0,1;\ 0,4]$, con $a = 1$. Para el tercer elemento, supongamos que en $j = 0$ se genera el candidato 0,2 y se asigna a h_0. Como $0,2 < 1,2$, la firma pasa a $[0,2;\ 0,1;\ 0,4]$.

Ahora las tres partes enteras son 0, de modo que $a = 0$. La iteración siguiente sería $j = 1$, pero $j > a$; el algoritmo omite $j = 1$ y $j = 2$ para este elemento.

5. ¿Dónde está el ahorro?

Elemento procesado	Iteraciones realizadas	Iteraciones omitidas de un máximo de 3
Primero	3	0
Segundo	2	1
Tercero	1	2
Total ilustrativo	**6**	**3**

Un recorrido completo habría realizado 9 iteraciones en este ejemplo; el corte temprano deja 6. Al omitir iteraciones, se evitan generar números aleatorios, hacer intercambios de la permutación y probar candidatos que no pueden mejorar la firma. El ahorro depende de los valores aleatorios y de los datos; esta ejecución pequeña solo ilustra el mecanismo.

El histograma se actualiza cuando cambia una posición. Por eso, el algoritmo no necesita recorrer de nuevo todas las posiciones para encontrar a en cada iteración. Sí hay un costo inicial para crear las m posiciones y las estructuras auxiliares.

6. Qué no significa “mejorar”

- “Mejorar la firma” significa únicamente bajar el mínimo guardado en alguna posición; no significa que dos documentos se parezcan más.
- El algoritmo no compara primero las partes enteras de dos firmas. Para estimar Jaccard se comparan los valores completos, posición por posición.
- Las posiciones de la firma no son los *buckets* de LSH (*Locality-Sensitive Hashing*), que se usan después para agrupar candidatos.

Idea central: SuperMinHash mantiene los mínimos de la firma y usa un histograma incremental para saber cuándo los candidatos restantes ya no pueden reducir ninguno de ellos.